Reflected XSS into a JavaScript String with Angle Brackets and Double Quotes HTML-Encoded and Single Quotes Escaped

1. Overview

Field	Details
Vulnerability	Reflected Cross-Site Scripting (XSS)
Category	Client-Side Injection
Injection Context	JavaScript String
Severity	Context-dependent
Lab Platform	PortSwigger Web Security Academy
Vulnerability Type	Reflected XSS

2. Description

Reflected Cross-Site Scripting (XSS) occurs when an application includes user-controlled input in an HTTP response without applying sufficient output encoding.

In this lab, user input is reflected inside a JavaScript string embedded within an HTML document.

The application applies HTML encoding to angle brackets and double quotes and escapes single quotes. However, these protections do not necessarily prevent JavaScript injection.

If an attacker can escape the intended JavaScript string context using an input sequence that is not properly neutralized, they may be able to inject executable JavaScript.

3. Root Cause

The vulnerability is caused by insufficient context-aware output encoding within a JavaScript string.

The application relies on character-level filtering instead of safely handling data according to the JavaScript parsing context.

HTML encoding does not automatically provide complete protection against JavaScript injection.

The security weakness arises when the application assumes that escaping selected characters is sufficient to make untrusted input safe.

4. Proof of Concept (PoC)

Testing environment: Authorized PortSwigger Web Security Academy lab.

1. Open the assigned lab.
2. Identify the parameter that is reflected into a JavaScript string.
3. Submit a harmless test value.
4. Inspect the HTTP response and locate the JavaScript string containing the reflected input.
5. Analyze the encoding applied to angle brackets, double quotes, and single quotes.
6. Examine whether the application handles other JavaScript-sensitive characters correctly.
7. In the authorized lab, test whether the string context can be escaped.
8. Verify the result with a harmless proof-of-execution statement.

Example vulnerable context:

```
None
var searchTerm = 'USER_INPUT';
```

This example demonstrates the JavaScript string context. It is not a confirmed representation of the lab's exact response.

5. Security Impact

If successfully exploited, reflected XSS may allow an attacker to:

- Execute JavaScript in the context of the vulnerable website.
- Manipulate the page displayed to a victim.
- Perform unauthorized actions through the victim's active session.
- Access sensitive information available to client-side scripts.
- Redirect users to attacker-controlled websites.
- Deliver deceptive content through a trusted application.

Exploitation generally requires a victim to access a specially crafted URL or otherwise trigger the vulnerable request.

6. Recommended Remediation

- Avoid placing untrusted input directly inside executable JavaScript.
- Use safe JSON serialization when transferring server-side data to client-side code.
- Apply JavaScript string encoding appropriate to the exact output context.
- Prefer non-executable data attributes or separate JSON responses.
- Use secure templating frameworks with contextual output encoding.
- Avoid relying on manual character replacement or blacklist filtering.
- Implement a restrictive Content Security Policy (CSP).
- Add automated tests for quotes, backslashes, line terminators, and HTML-sensitive characters.

7. Verification

After implementing the remediation:

1. Repeat the original test cases.
2. Inspect the generated HTML and JavaScript.
3. Confirm that reflected input remains data rather than executable code.
4. Test special-character and encoding variations.
5. Verify that legitimate application functionality remains operational.

8. References

- PortSwigger: Reflected XSS
- PortSwigger: XSS Contexts
- OWASP: Cross Site Scripting Prevention Cheat Sheet

9. Disclaimer

This report is intended for educational purposes and authorized security assessments. Testing should only be performed in approved environments or against systems with explicit authorization.